

KU LEUVEN

Geavanceerde computerarchitectuur

Getting acquainted with GPU programming

Jarno De Jonckheere

22nd February, 2024

1 Verslag

In deze labosessie was het de bedoeling om te leren werken met de CUDA programming framework in combinatie met C++. Tot dit einde was de opdracht om een programma te schrijven die een array flipt.

De CUDA programming framework zorgt ervoor dat we het architecturaal voordeel van de GPU, voor parralelliseerbare opdrachten, ten opzichte van de CPU kunnen benutten die eerder geoptimaliseerd is voor seriële bewerkingen.
Om ervoor te zorgen dat we hier geen last hebben van race conditions zullen we twee verschillende arrays aanmaken zodanig dat verschillende threads niet proberen te lezen/ schrijven naar hetzelfde geheugen op hetzelfde moment.

Arrays die worden aangemaakt op de CPU (Host) moeten eerst en vooral gemigreerd worden naar de GPU (Device) vooraleer we deze met de GPU trachten te benaderen. Nadat de bewerkingen met de GPU voltooid zijn migreren we het geheugen terug naar de CPU. zoals in elk C++ programma moet dynamisch geheugen zelf vrijgemaakt worden. dit moet hier gedaan worden voor zowel device memory als host memory.

```
int numElements = 50000;
size_t size = numElements * sizeof(float);

float *h_A = (float *)malloc(size); // memory allocation on host

for (int i = 0; i < numElements; ++i) {
    h_A[i] = rand() / (float)RAND_MAX;
} // initialisation on host

float *d_A = NULL;
err = cudaMalloc((void **)&d_A, size); // allocation on device


err = cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
// migration host to device

err = cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
// example of migration from device to host


err = cudaFree(d_A ); // for freeing on device
free(h_A); // for freeing on host
```

Het oproepen van een GPU-functie wordt gedaan met een oproep van de vorm:

```
vectorFlip<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_C, numElements);
```

Wanneer het aantal threads per block op voorhand ingesteld is wordt het aantal blocks per grid berekend als:

```
int blocksPerGrid = (numElements + threadsPerBlock - 1) / threadsPerBlock;
```

het kan hier optreden dat het totale aantal threads groter is dan het nodige aantal threads, maar dit kan worden opgelost met behulp van een simpele conditionele expressie, zodanig dat sommige threads idle zullen blijven. Vervolgens wordt aan elke thread een element van de oosrspronkelijke array gelinkt en wordt de array geflipt.

```
1     __global__ void vectorFlip(const float *A, float *C,int numElements) {
2   int i = blockDim.x * blockIdx.x + threadIdx.x;
3
4   if (i < numElements) {
5     C[i] = A[numElements-1-i];
6   }
7 }
```